

Arduino + L298N + DC Motor

Introduction to DC Motors

Basic Parameters

Rated Voltage: The rated voltage is the voltage required for the DC motor to operate at its optimal performance.

Speed: Speed is usually expressed in RPM (Revolutions Per Minute). There is an inverse relationship between speed and torque. In motor applications, a balance between the two must be achieved to meet specific usage needs.

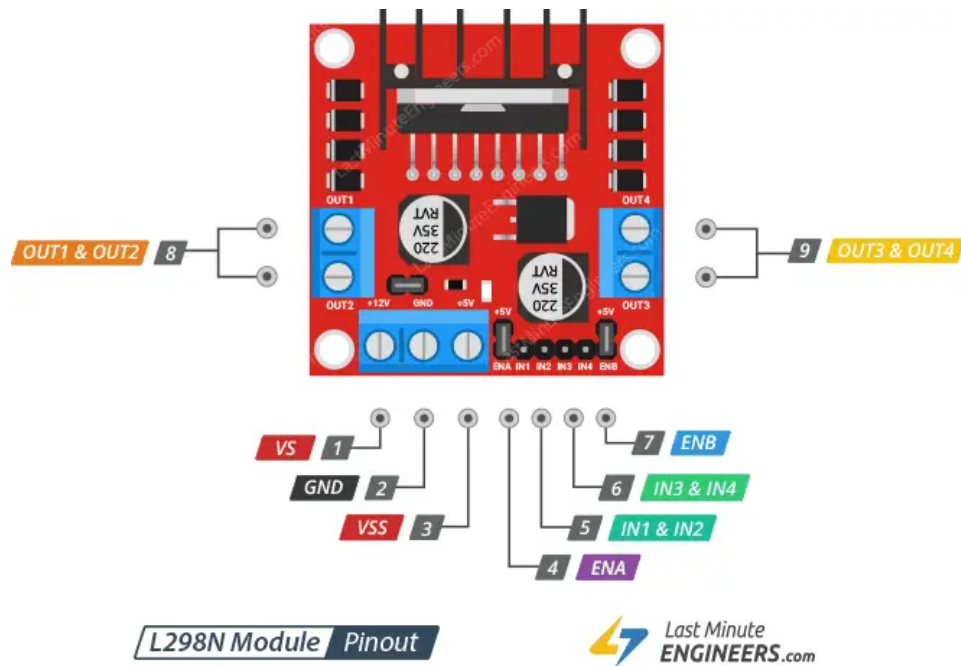
For the VG100-S2 first stage project, the choice of rated voltage for the motor must be less than or equal to 12V. A lower speed can be chosen to increase torque.

Working Principle

The two electrodes of the DC motor are connected to high and low voltages, respectively. Different voltage differences result in different speeds.

L298N Motor Driver

Interface Introduction



- OUT1, OUT2, OUT3, OUT4: Current output, connected to the DC motor
- 12V (VS), and GND: Used for current input (does not necessarily need a 12V power supply. 5V~35V is also acceptable)
- 5V (VSS), can be used for 5V output
- IN1, IN2, IN3, IN4: Signal input, connected to Arduino
- EN1, EN2: Enable inputs for side A and side B, can be used to control motor speed

It can provide a maximum current of approximately 2A.

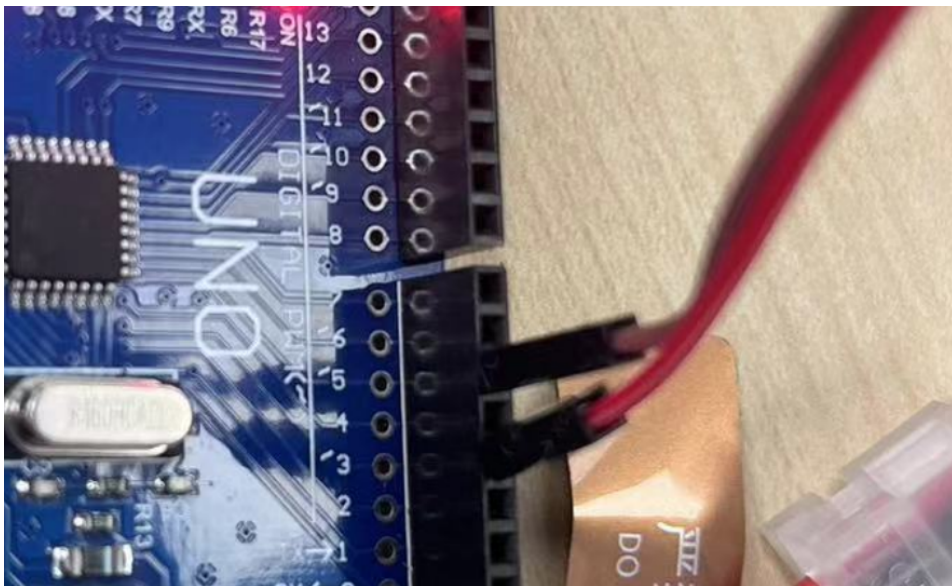
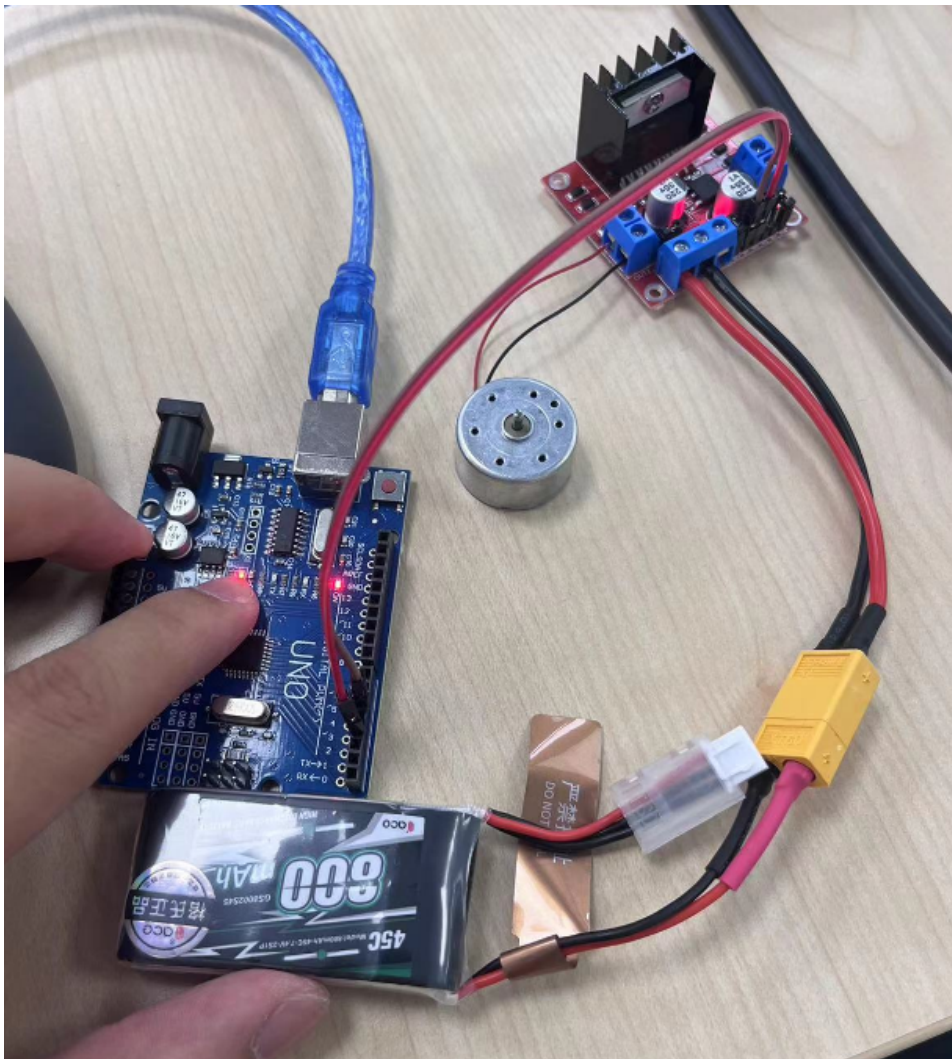
Method 1 Usage

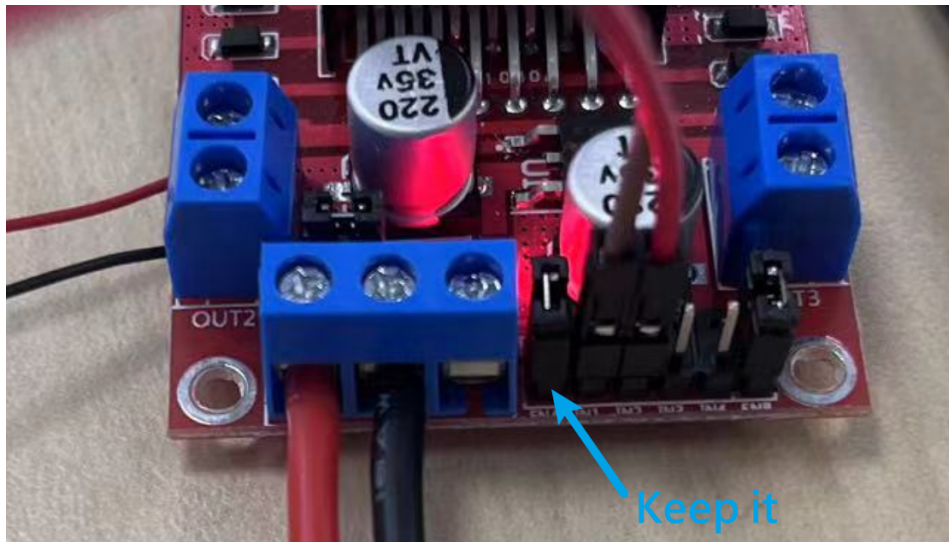
IN1 and IN2 control the motor direction and speed.

Requires two analog output pins.

The implementation idea is simpler.

Wiring





1. Connecting Arduino to L298N Driver Module

- **IN1**: Connect Arduino pin D5 to L298N IN1 pin.
- **IN2**: Connect Arduino pin D3 to L298N IN2 pin.
- Do not remove the jumper cap from ENA.

2. Power Connection

- **Driver Module Power Input:**
 - Connect the **VCC** pin of the L298N to the battery.
 - Connect the **GND** pin of the L298N to the negative pole (ground) of the power supply.
- **Arduino Power:**
 - Power can be supplied via USB or an external power source.

3. Motor Connection

- Connect the two wires of the DC motor to the **OUT1** and **OUT2** pins of the L298N. Ensure the motor is connected correctly so that IN1 and IN2 can control the forward and reverse rotation of the motor.

Code

```
int IN1 = 5; // D5 pin connected to the IN1 of the driver module
int IN2 = 3; // D3 pin connected to the IN2 of the driver module

void motor(int speed1) { // speed1 is the speed to drive the motor, value
range -255 to 255, positive for forward, negative for reverse
    int s1, s2; // Define s1 and s2 variables to indicate forward and reverse
state of the motor
    if(speed1 > 0) {
        s1 = IN1; // When forward, s1 indicates IN1
        s2 = IN2; // When forward, s2 indicates IN2
    } else {
        s1 = IN2; // When reverse, s1 indicates IN2
        s2 = IN1; // When reverse, s2 indicates IN1
        speed1 = -speed1; // Take the absolute value of speed1 to make it
positive
    }
}
```

```
}  
digitalWrite(s1, LOW); // Write LOW to the pin represented by s1 to  
determine the negative pole of the motor  
analogWrite(s2, speed1); // Write an analog signal to the pin represented  
by s2 to control the motor speed according to the absolute value of speed1  
}  
  
void setup() {  
  pinMode(IN1, OUTPUT); // Set D5 as output pin  
  pinMode(IN2, OUTPUT); // Set D3 as output pin  
  Serial.begin(9600);  
}  
  
void loop() {  
  motor(50); // Call motor function to set motor speed to 50, motor rotates  
forward  
  delay(1000); // Wait for 1 second to keep the motor at the current speed  
  motor(100); // Call motor function to set motor speed to 100, motor  
rotates forward  
  delay(1000); // Wait for 1 second to keep the motor at the current speed  
  motor(255); // Call motor function to set motor speed to 255 (maximum  
speed), motor rotates forward  
  delay(1000); // Wait for 1 second to keep the motor at the current speed  
  motor(-100); // Call motor function to set motor speed to -100, motor  
rotates backward  
  delay(1000); // Wait for 1 second to keep the motor at the current speed  
}  
}
```

Function Introduction

digitalWrite

- **Function:** `digitalWrite` is used to set a digital pin to HIGH or LOW.
- **Usage:** `digitalWrite(pin, value);`
 - `pin`: The pin number to operate on (e.g., D5).
 - `value`: `HIGH` or `LOW`, representing high and low levels, respectively.
- **Pin Requirement:** Can be used on any pin.

Example:

```
[  
  
digitalWrite(5, HIGH); // Set D5 pin to HIGH  
digitalWrite(3, LOW);  // Set D3 pin to LOW  
  
]
```

analogWrite

- **Function:** `analogWrite` is used to output an analog signal to a PWM pin, ranging from 0 to 255, simulating an analog signal through PWM technology.
- **Usage:** `analogWrite(pin, value);`
 - `pin`: The PWM pin number to operate on (e.g., D6).
 - `value`: A value between 0 and 255, where 0 represents 0% duty cycle (LOW), and 255 represents 100% duty cycle (HIGH).
- **Pin Requirement:** Can only be used on pins that support PWM.

Example:

```
analogWrite(6, 128); // Output a 50% duty cycle PWM signal on D6 pin
```

Method 2 Usage

IN1 and IN2 control the motor direction.

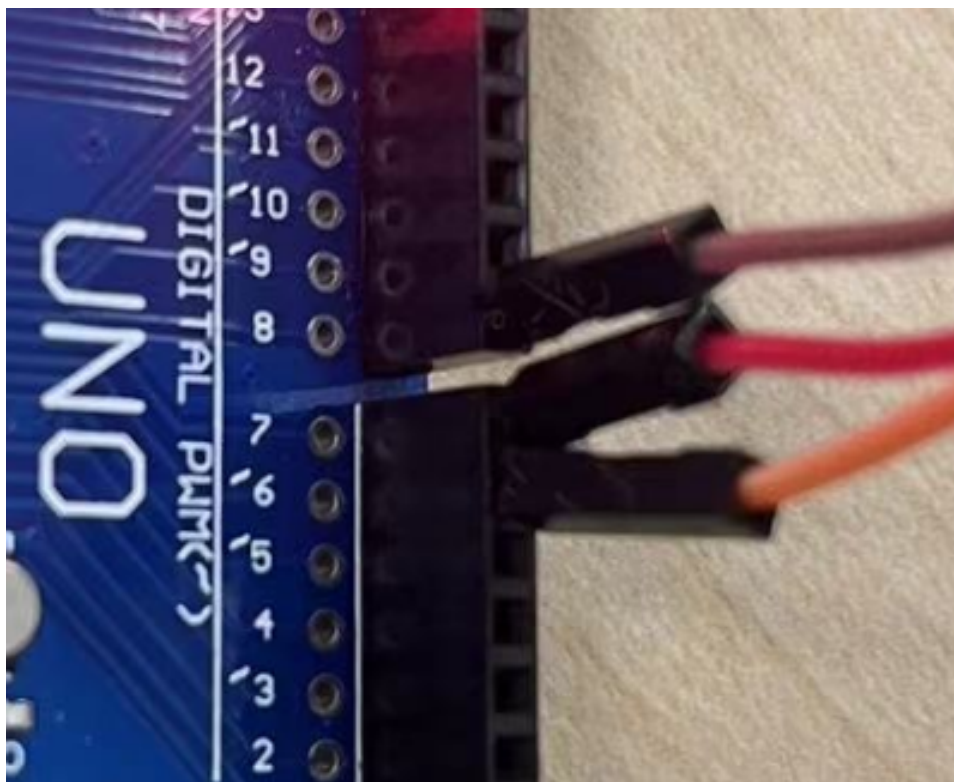
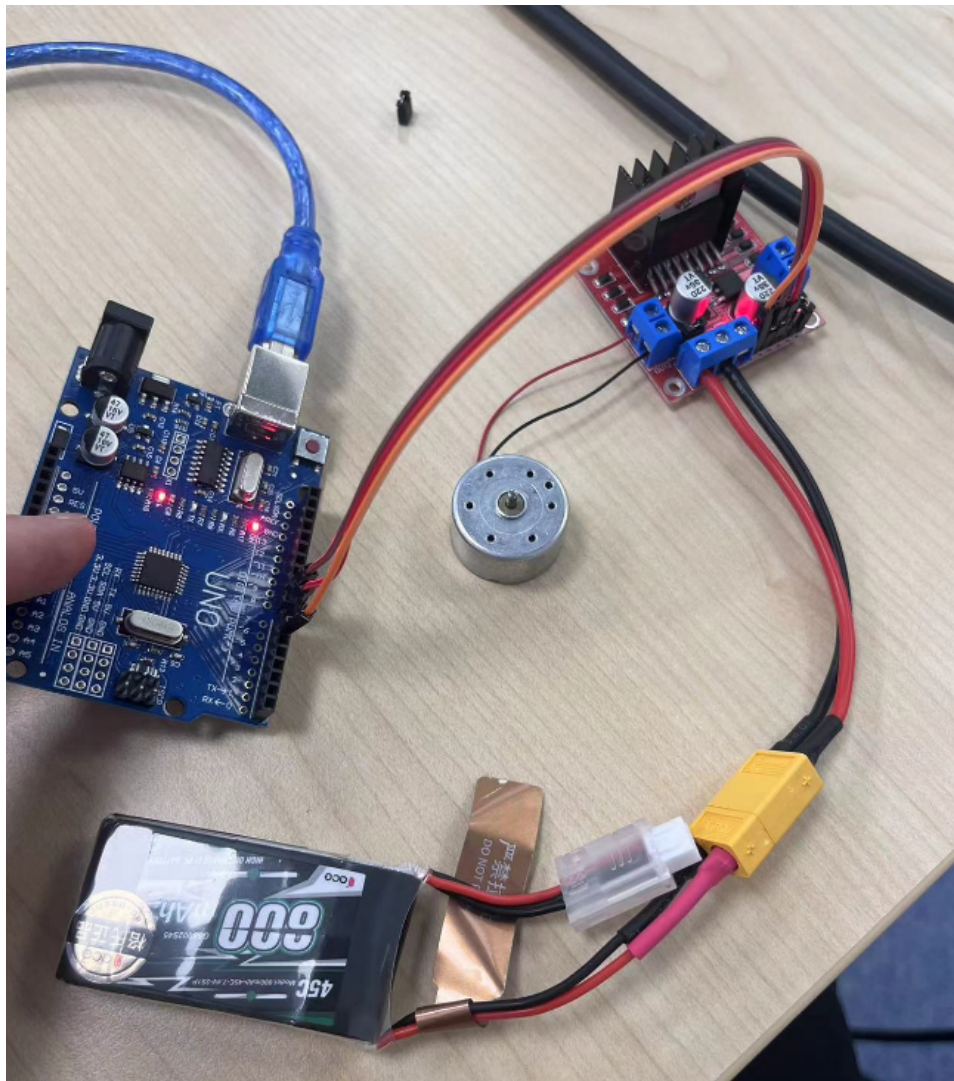
ENA controls the motor speed.

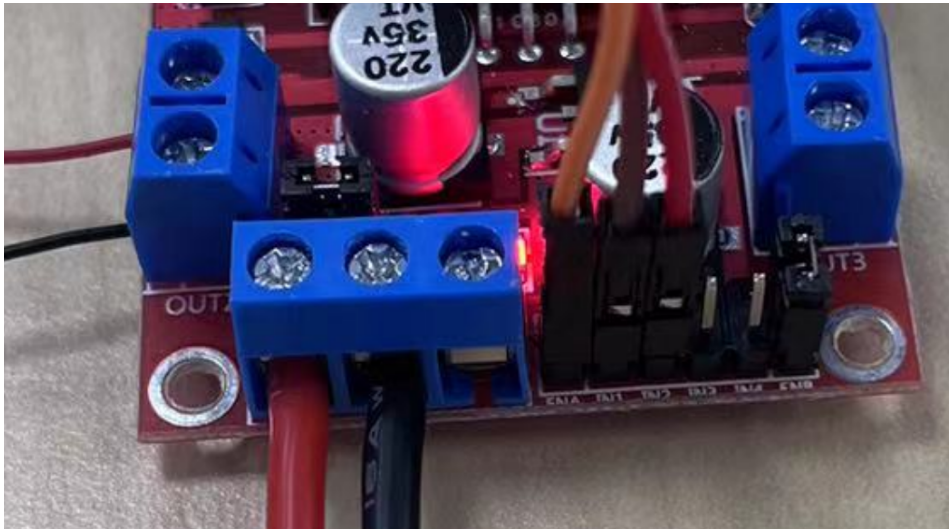
Requires two digital output pins + one analog output pin.

Saves analog pins, code is more elegant, wiring is a bit more complicated.

Separately control forward/reverse rotation and speed.

Wiring





1. Connecting Arduino to L298N Driver Module

- **IN1**: Connect Arduino pin D5 to L298N IN1 pin.
- **IN2**: Connect Arduino pin D3 to L298N IN2 pin.
- **EN1**: Connect Arduino pin D6 to L298N EN1 pin.

2. Power Connection

- **Driver Module Power Input:**
 - Connect the **VCC** pin of the L298N to the battery.
 - Connect the **GND** pin of the L298N to the negative pole (ground) of the power supply.
- **Arduino Power:**
 - Power can be supplied via USB or an external power source.

3. Motor Connection

- Connect the two wires of the DC motor to the **OUT1** and **OUT2** pins of the L298N. Ensure the motor is connected correctly so that IN1 and IN2 can control the forward and reverse rotation of the motor.

Code

C style

```
int IN1 = 8; // D8 pin connected to the IN1 of the driver module
int IN2 = 7; // D7 pin connected to the IN2 of the driver module
int EN1 = 6; // D6 pin connected to the EN1 of the driver module, used to
control motor speed

void motor(int speed1) { // speed1 is the speed to drive the motor, value
range -255 to 255, positive for forward, negative for reverse
    if(speed1 > 0) {
        digitalWrite(IN1, HIGH); // IN1 high level, motor rotates forward
        digitalWrite(IN2, LOW);  // IN2 low level
    } else {
        digitalWrite(IN1, LOW);  // IN1 low level
        digitalWrite(IN2, HIGH); // IN2 high level, motor rotates backward
        speed1 = -speed1; // Take the absolute value of speed1 to make it
```



```

    positive
    }
    analogWrite(EN1, speed1); // Use PWM signal to control motor speed
}

void setup() {
    pinMode(IN1, OUTPUT); // Set D8 as output pin
    pinMode(IN2, OUTPUT); // Set D7 as output pin
    pinMode(EN1, OUTPUT); // Set D6 as output pin
    Serial.begin(9600); // Initialize serial communication, set baud rate to
9600
}

void loop() {
    motor(50); // Call motor function to set motor speed to 50, motor rotates
forward
    delay(1000); // Wait for 1 second to keep the motor at the current speed
    motor(100); // Call motor function to set motor speed to 100, motor
rotates forward
    delay(1000); // Wait for 1 second to keep the motor at the current speed
    motor(255); // Call motor function to set motor speed to 255, motor
rotates forward
    delay(1000); // Wait for 1 second to keep the motor at the current speed
    motor(-100); // Call motor function to set motor speed to -100, motor
rotates backward
    delay(1000); // Wait for 1 second to keep the motor at the current speed
}
}

```

C++ style

```

[
class Motor {
private:
    int IN1; // IN1 pin of the motor driver module
    int IN2; // IN2 pin of the motor driver module
    int EN1; // EN1 pin of the motor driver module, used to control motor
speed

public:
    Motor(int in1, int in2, int en1) { // Constructor to initialize the
pins
        IN1 = in1;
        IN2 = in2;
        EN1 = en1;

        pinMode(IN1, OUTPUT); // Set IN1 as output pin

```

```
pinMode(IN2, OUTPUT); // Set IN2 as output pin
pinMode(EN1, OUTPUT); // Set EN1 as output pin
}

void setSpeed(int speed) { // Method to set motor speed, speed range
from -255 to 255
    if (speed > 0) {
        digitalWrite(IN1, HIGH); // IN1 high level, motor rotates forward
        digitalWrite(IN2, LOW); // IN2 low level
    } else {
        digitalWrite(IN1, LOW); // IN1 low level
        digitalWrite(IN2, HIGH); // IN2 high level, motor rotates backward
        speed = -speed; // Take the absolute value of speed to make it
positive
    }
    analogWrite(EN1, speed); // Use PWM signal to control motor speed
}
};

Motor motor1(8, 7, 6); // Create an instance of the Motor class, specifying
IN1, IN2, EN1 pins

void setup() {
    Serial.begin(9600); // Initialize serial communication, set baud rate to
9600
}

void loop() {
    motor1.setSpeed(50); // Set motor speed to 50, motor rotates forward
    delay(1000); // Wait for 1 second to keep the motor at the current speed
    motor1.setSpeed(100); // Set motor speed to 100, motor rotates forward
    delay(1000); // Wait for 1 second to keep the motor at the current speed
    motor1.setSpeed(255); // Set motor speed to 255, motor rotates forward
    delay(1000); // Wait for 1 second to keep the motor at the current speed
    motor1.setSpeed(-100); // Set motor speed to -100, motor rotates backward
    delay(1000); // Wait for 1 second to keep the motor at the current speed
}
}
```

CN_ver

[See Chinese version here](#)

References

-
- [Arduino L298N Guide | https://nju-robot-club.gitbook.io/arduino/week4/l298n](https://nju-robot-club.gitbook.io/arduino/week4/l298n)
 - [L298N Motor Driver Tutorial | https://lastminuteengineers.com/l298n-dc-stepper-driver-arduino-tutorial/](https://lastminuteengineers.com/l298n-dc-stepper-driver-arduino-tutorial/)